МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2
по курсу «Операционные системы»

Процессы и каналы
Вариант 22

Выполнил: Яковлев Артём Сергеевич
Группа: М8О-203БВ-25
Преподаватель: Миронов Евгений Сергеевич

Москва, 2026

Цели и условие

Цель работы – научиться запускать дочерние процессы и передавать им данные через каналы[1].

Программа сначала получает от пользователя имена двух файлов. Затем родительский процесс запускает двух детей. Каждому ребёнку он может отправлять строки через отдельный канал. Ребёнок переворачивает полученную строку и записывает результат в свой файл.

По варианту 22 родитель выбирает первого ребёнка с вероятностью 80%, а второго – с вероятностью 20%. Программа должна работать на POSIX-системах и Windows.

Программное и аппаратное обеспечение

- Язык программы – C (стандарт C11); для сборки используется CMake.
- Локальная проверка выполнена на macOS, компьютер ARM64, компилятор Apple Clang.
- Сборка на Windows настроена в GitHub Actions на `windows-2022`.
- Для проверки результата используется небольшой тест на Python.

Метод решения

Программа состоит из родительской программы `lab2_parent` и дочерней `lab2_child`. Родитель запускает два экземпляра дочерней программы. Для каждого он создаёт канал (pipe) и файл.

Работа программы проходит так:

1. Родитель читает имена двух файлов и запускает обоих детей.
2. Для каждой новой строки он получает число от 0 до 99. Если число меньше 80, строка отправляется первому ребёнку; иначе – второму.
3. Ребёнок читает строку, переворачивает её и записывает в свой файл. Например, `"Hello"` превращается в `"olleH"`.
4. Когда строки заканчиваются, родитель закрывает каналы и ждёт завершения детей.

Длинные строки читаются с помощью увеличиваемого буфера. Для работы с процессами используются разные функции POSIX и Windows, но порядок действий остаётся одним и тем же. В Windows-реализации путь к программе и имена файлов должны состоять из ASCII-символов; содержимое строк остаётся в UTF-8.

Описание программы

Основные файлы проекта:

- `main.c` – родитель: читает ввод, выбирает ребёнка и отправляет строки.
- `child.c` – ребёнок: переворачивает строки и выводит результат.
- `line_io.c` – читает строки без заранее заданного ограничения длины.
- `process_posix.c` и `process_win.c` – создают каналы и запускают процессы в соответствующей операционной системе.
- `process.h` – описывает структуру `ChildProcess`, в которой родитель хранит данные о запущенном ребёнке и канале.
- `CMakeLists.txt` – выбирает нужный файл для текущей системы и собирает обе программы.
- `tests/test.py` проверяет результат работы для Windows.

Если при создании файла, канала или процесса происходит ошибка, программа сообщает о ней и закрывает уже открытые ресурсы.

Результаты

На macOS программа прошла проверку. В одном запуске были введены строки `Hello`, «Привет», `abcdef` и `12345`. В файле `first.txt` оказались `olleH`, «тевирП» и `fedcba`; в `second.txt` — `54321`. Все строки записаны по одному разу и перевёрнуты правильно. В другом запуске распределение по файлам может отличаться, потому что ребёнок выбирается случайно.

Опубликованная версия проекта находится в репозитории[2]. Её сборка и тест на Windows также прошли успешно в GitHub Actions[3].

Выводы

В ходе работы написана программа, в которой родитель передаёт строки двум дочерним процессам через отдельные каналы. Дети переворачивают строки и сохраняют их в разные файлы. Такая схема подходит для задач, где поток независимых сообщений нужно распределить между несколькими процессами. Например, каждый процесс может обрабатывать свою часть входных данных и записывать результат в отдельный файл.

Основные трудности в реализации возникли при запуске дочерних процессов и работе с каналами: важно передать ребёнку нужный конец канала и вовремя закрыть ненужные концы. Иначе ребёнок может продолжать ждать ввод после того, как родитель закончил отправлять строки. Для Windows также потребовался отдельный код запуска процессов.

Проверка на macOS и Windows показала правильную работу на латинских и русских строках.

Список литературы

[1] Лабораторная работа №2 по курсу «Операционные системы». Условие задания, вариант 22.

[2] F14my. «OS-lab2». Исходный код.

[3] GitHub Actions. «Windows test». Успешный запуск для коммита 7193d5b

[4] Microsoft Learn. Creating a Child Process with Redirected Input and Output. Документация Win32 API.